Computer Vision p2

2026-08-2

Hebah Omer Soleman

Table of Contents

DataLoaders

- As we have previously established that Deep Learning has been made possible by large amount of data and computational resourse
- An important aspect to keep in mind is the data handling:
  - How do we handle large amounts of data?
  - How to we read different components of data (from possible different parts of our hard drive) and provide it to our training algorithms?
  - How do we feed this data to SGD algorthims in a streamlined manner?
- PyTorch provides Dataset and DataLoaders to handle data in an efficient manner.
- We will extend the Dataset and DataLoaders class to construct our own Dataloaders

DataLoaders (cont.)

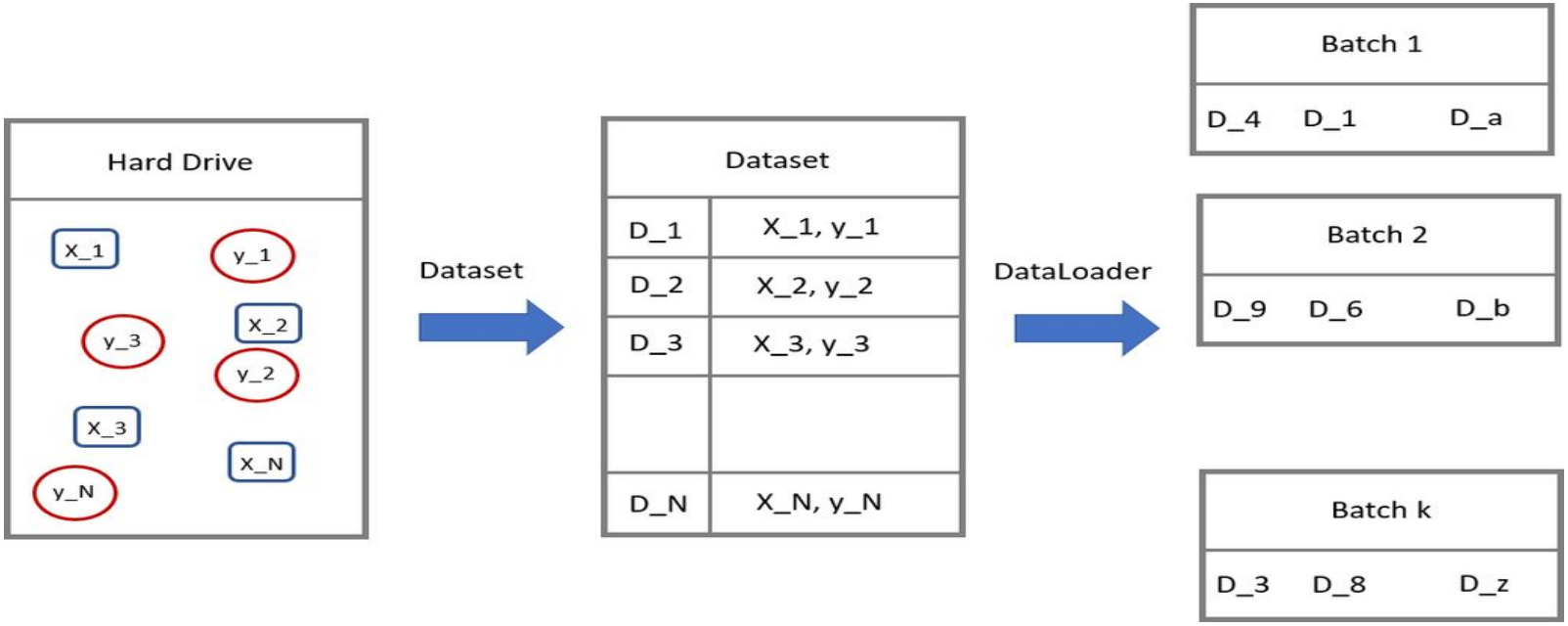
- Datasets in PyTorch: The torch.utils.data.Dataset class is an abstract base class used to define and manage datasets for training and evaluation.
- Custom Dataset Creation: You can create a custom dataset by subclassing Dataset and implementing

  len () (returns dataset size) and

  getitem () (fetches a single data sample).

- DataLoaders for Batching: torch.utils.data.DataLoader is used to load data in mini-batches, shuffle data, and utilize multiprocessing for efficiency.
- Built-in Datasets: PyTorch provides datasets in torchvision.datasets and torchtext.datasets, making it easy to load common datasets like MNIST, CIFAR-10, and IMDB.

DataLoaders (cont.)



Data Augmentation

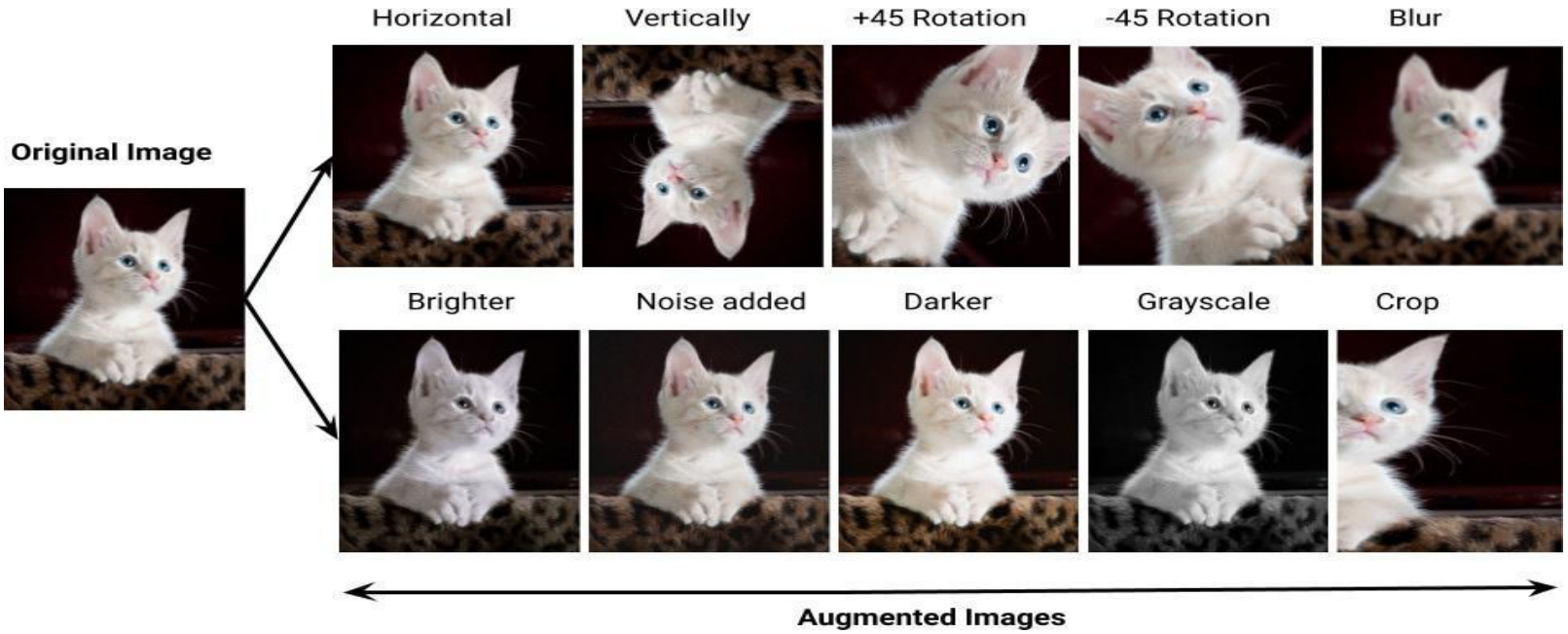
- Data is the fundamental building block of any machine learning algorithm
- In several applications we don’t have access to unlimited data
- So we use Data Augmentation techniques to improve the preformance of our models
- Note: It is better to spend time on data rather than fine-scale architecture search in deep learning

Data Augmentation (cont.)

Create virtual training samples

- Horizontal flip
- Random crop
- Color casting
- Geometric distortion
- Translation
- Rotation

Data Augmentation (cont.)



[0]https://pranjal-ostwal.medium.com/data-augmentation-for-computer-vision-b88b818b6010

Data Augmentation (cont.)

[0]To simplify data augmentation, tools like https://github.com/albumentations-team/albumentations

Data Augmentation (cont.)

But, there are many types of augmentations. How do we choose the right ones for our task?

Data Augmentation (cont.)

- But, there are many types of augmentations. How do we choose the right ones for our task
- **Answer: Error Analysis** – Identify model weaknesses and apply augmentations that address those issues.

Error Analysis for Data Augmentation

Steps:

1. Train a baseline model.
2. Make predictions on validation data.
3. Inspect the worst predictions to identify model weaknesses.
4. Apply relevant augmentations to address these issues.

Examples:

- **Failure with small objects** → Use *Scale Augmentation*.
- **Failure with different colors/environments** → Use *Color Augmentations*.
- **Failure with rotated images** → Use *Rotation Augmentations*.
- **Failure with blurry images** → Use *Noise Augmentations*.
- . . . etc.

Data Augmentation

Note: Data augmentation is applied only to the training data.
Applying it to validation/test data **directly** would lead to **incorrect evaluation and misleading performance metrics**.

Data Augmentation

However, there is a special inference technique called **Test-Time Augmentation (TTA)** where multiple augmented versions of the same image are passed through the model separately, and the predictions are averaged to improve accuracy.

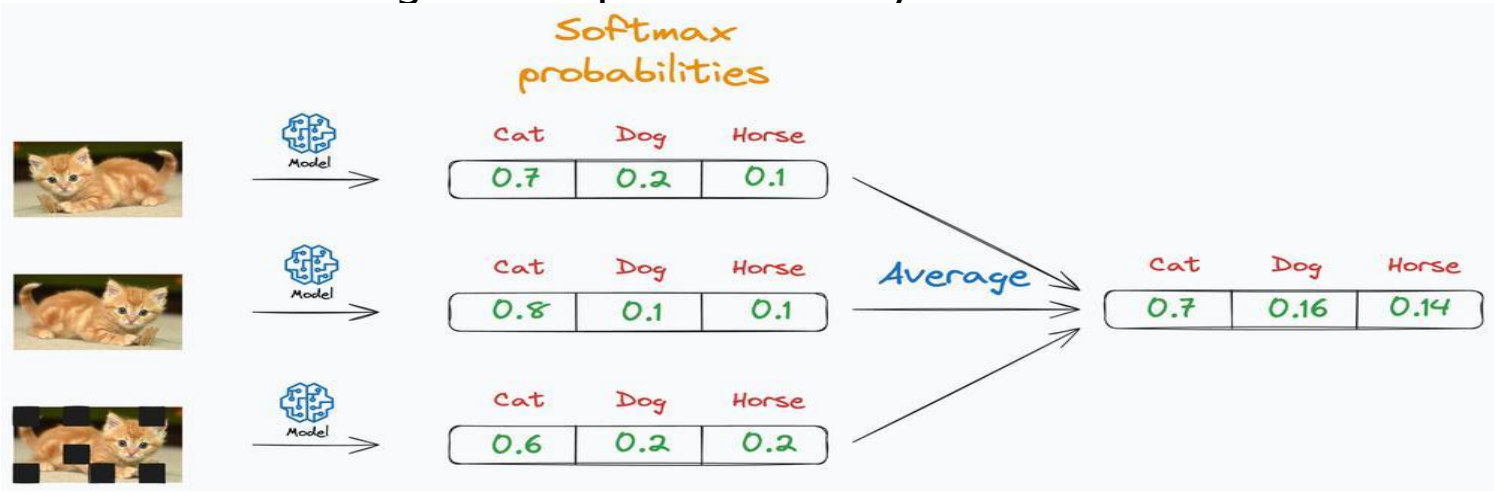


Figure 2: Test-time Augmentation

Data Augmentation

TTA is an advanced technique.

Applying it requires **custom scripts** to generate augmented versions of test images, pass them through the model, and aggregate predictions.

Transfer Learning

What is Fine-Tuning?

- A strategy in **transfer learning** where a pre-trained model is adapted to a new task.
- Instead of training from scratch, we start from a model already trained on a related task.

Why use Fine-Tuning?

- Saves time and computational resources.
- Improves performance, especially with limited data.

When to fine-tune your model?

New dataset is small + similar distribution to original dataset:

- Freeze (or partially freeze) feature extraction layers and fine-tune the classifier.

New dataset is small + different distribution to original dataset:

- Use the pretrained network as a generic feature extractor and train a light classifier on top (e.g., SVM).
- In modern practice, you might also freeze earlier layers and selectively fine-tune later layers.

New dataset is large, regardless of the original data distribution:

- Fine-tune the entire network (both features extractor and classifier).

Finetuning

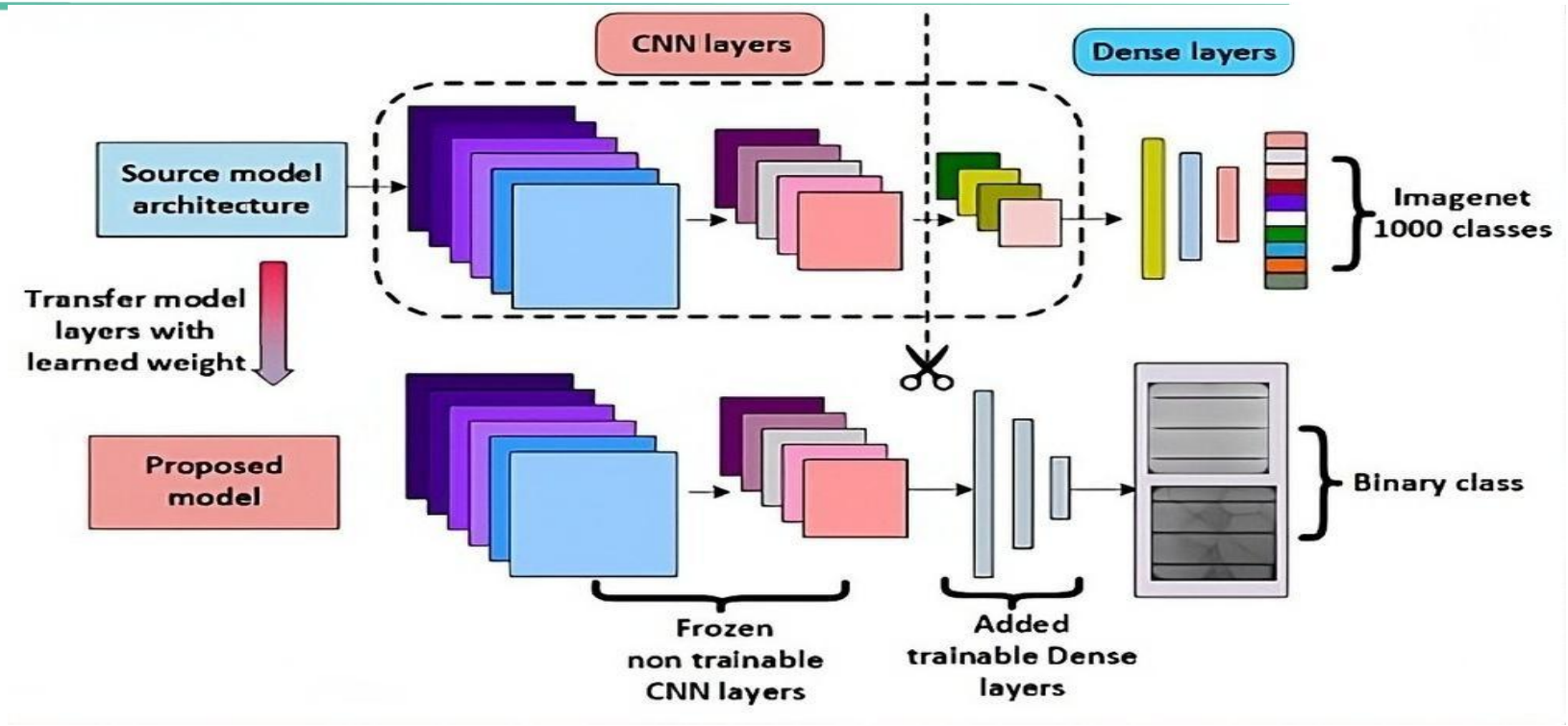


Figure 3: Learning and Transferring Mid-Level Image Representations using Convolutional Neural Networks

Ensembling

- No single model is perfect. Different models make different types of errors.
- Let's visualize the predictions of two different models:

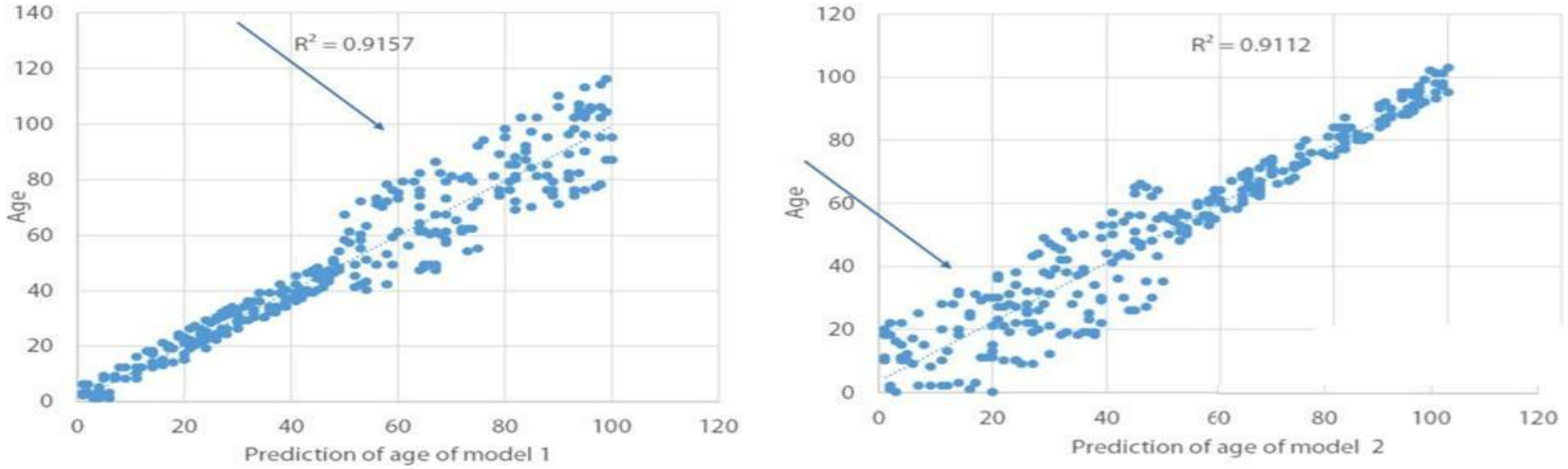


Figure 4: Predictions of Model 1 and Model 2

Ensembling

- Combining predictions of diverse models can reduce errors and improve accuracy. This technique is called **Ensembling**.

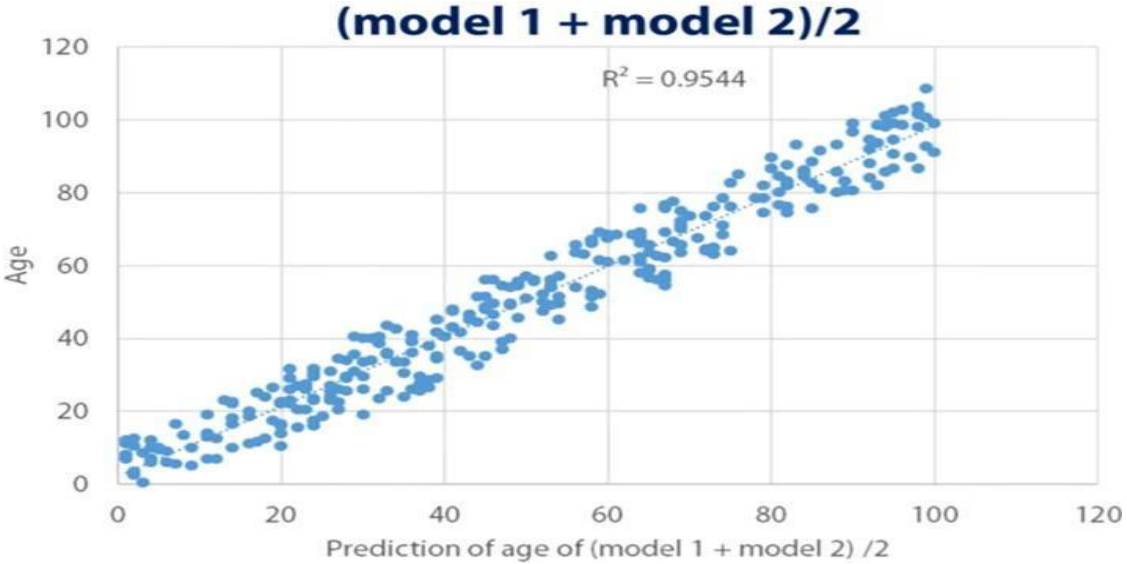


Figure 5: Averaged Predictions: Better Correlation

Ensembling

- Diverse models are key to effective ensembling. Here are two strategies:
  - **Bagging (Bootstrap Aggregating):** Train multiple models on different subsets of data (e.g., Random Forest model).

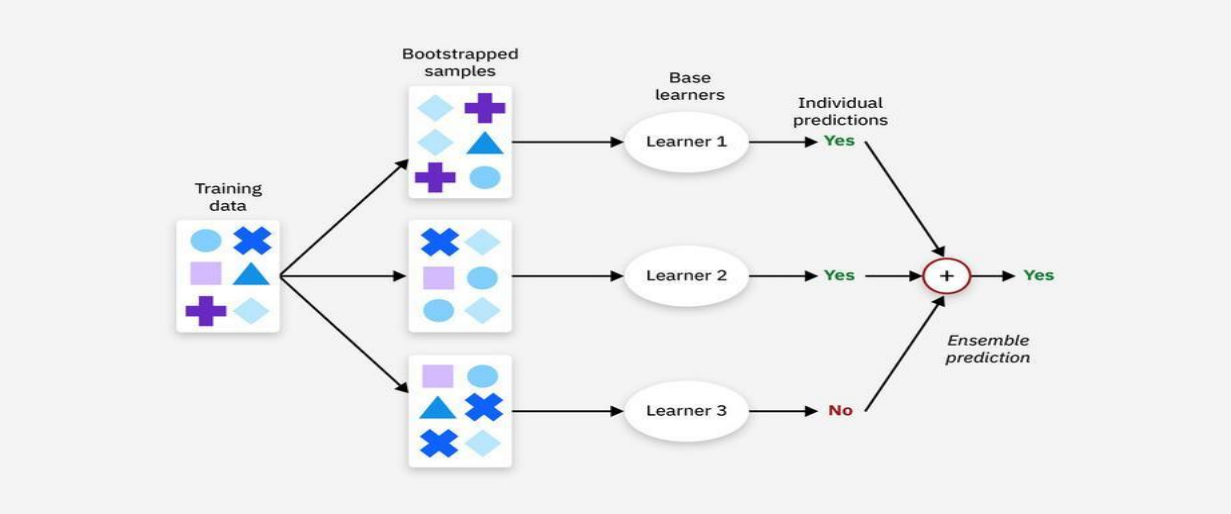


Figure 6: Bagging example

Ensembling

- **Boosting:** Train models sequentially, where each model focuses on the errors of the previous one (e.g., AdaBoost, Gradient Boosting).

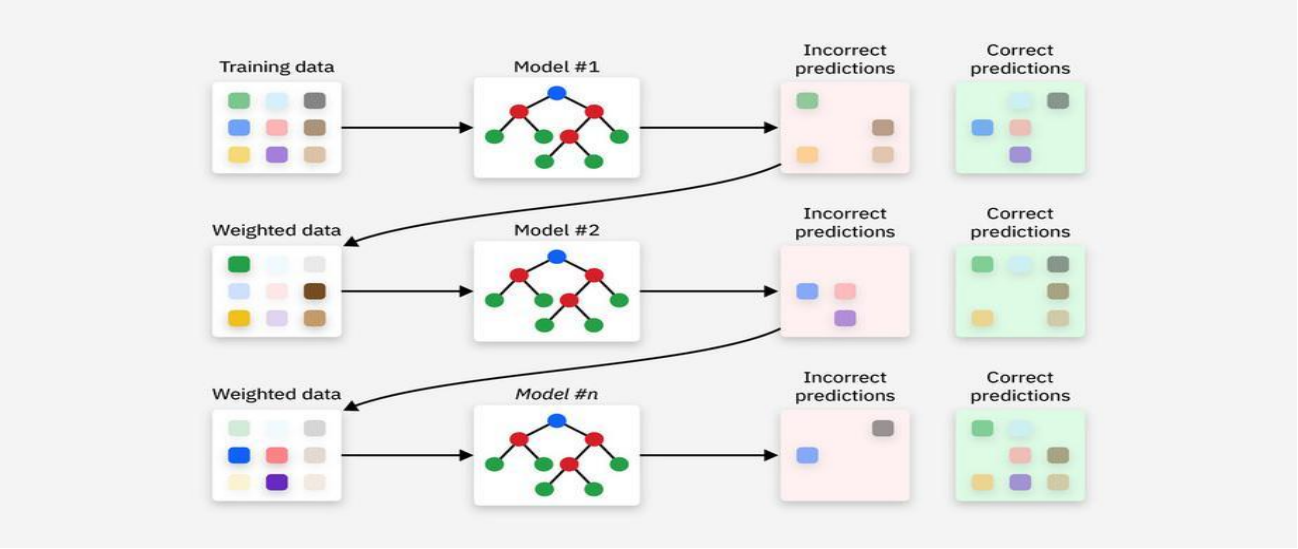


Figure 7: Boosting Example

Ensembling

- These stratigies introduce diversity in models, but how can we combine their predictions?

Ensembling

We can combine classifiers predictions using two ways:

- **Hard Voting:**
  - Each model votes for a class.
  - The class with the majority of votes is selected.
- **Soft Voting:**
  - Average the predicted probabilities from each model.
  - The class with the highest average probability is selected.

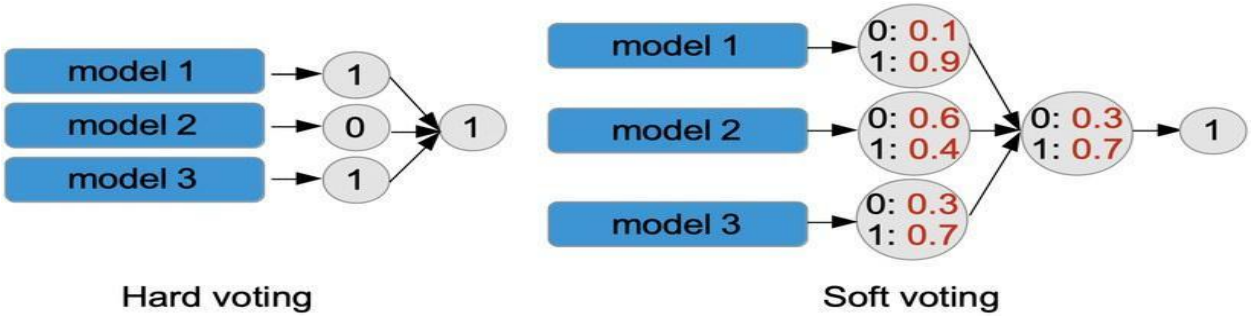


Figure 8: Illustration of Hard and Soft Voting for Classifiers

Ensembling

We can combine classifiers predictions using two ways:

- **Hard Voting:**
  - Each model votes for a class.
  - The class with the majority of votes is selected.
- **Soft Voting:**
  - Average the predicted probabilities from each model.
  - The class with the highest average probability is selected.

For regressors:

- Take the average of predictions from all models.

Ensembling

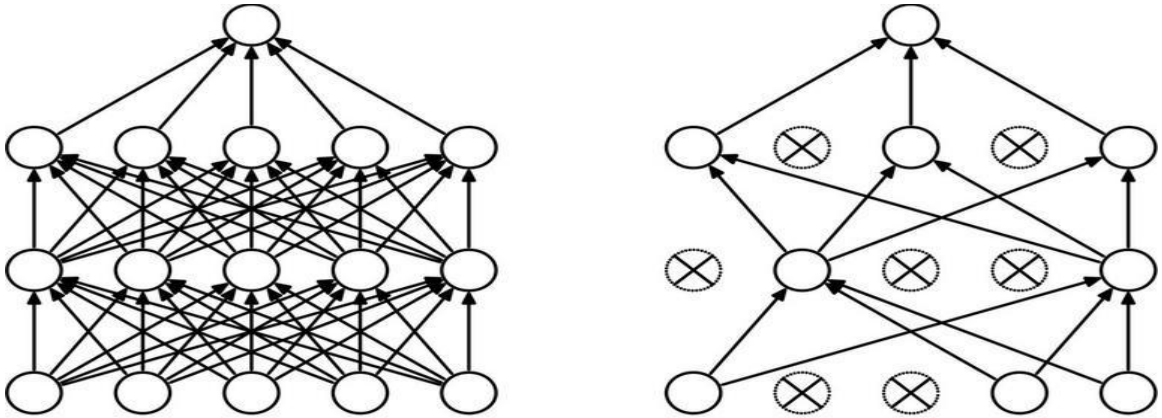
Team-work is the best policy.

We can train multiple networks for the same task then ensemble to get better results.

Dropout

Dropout is a regularization technique used to prevent overfitting in neural networks.

During training, it randomly drops (set to 0) a fraction of neurons in each layer based on a specified probability for every forward pass.



(a) Standard Neural Net

(b) After applying dropout.

[0]Srivastava JMLR 2014

Dropout

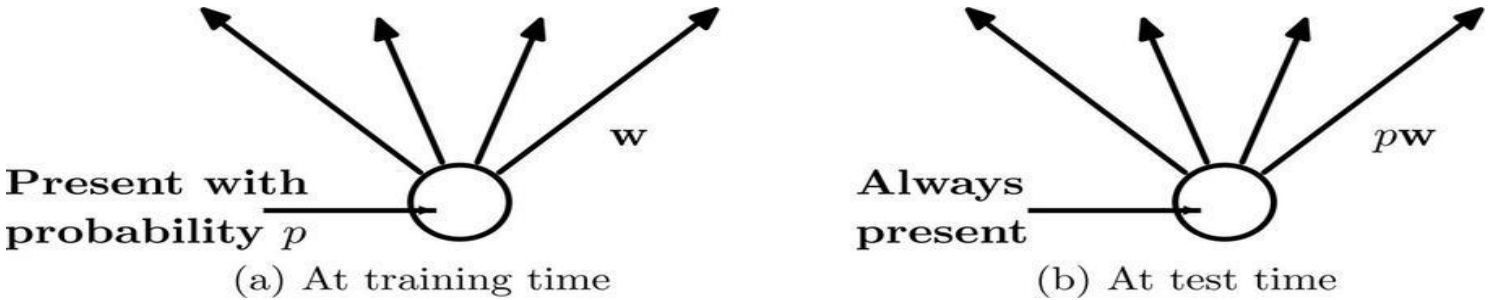
Why is dropping neurons randomly useful?

1. By dropping a subset of neurons during each forward pass, the network learns **not to rely too heavily on specific connections**, encouraging the network to use more connections.
2. Dropout acts as an **efficient ensemble** of multiple smaller networks, each trained on a random subset of neurons.
3. More Connections + Diverse Ensemble = **less overfitting**.

Dropout

We drop neurons during training, but what should we do during inference?

During inference: We use all neurons.



but...

Dropout

In PyTorch, dropout behavior is controlled by the model's mode:

```
# Training Mode:  Enables dropout model.train()
output = model(input)
# Evaluation Mode:  Disables dropout, scales activations model.eval()
output = model(input)
```

Batch Normalization

Key Insight: Networks train better when inputs are normalized

So why not normalize intermediate layers too?

Problem: forcing every layer to output zero-mean, unit-variance values might be too restrictive (not always optimal).

Solution: Let the network learn if it wants normalization!

Batch Normalization

During Training:

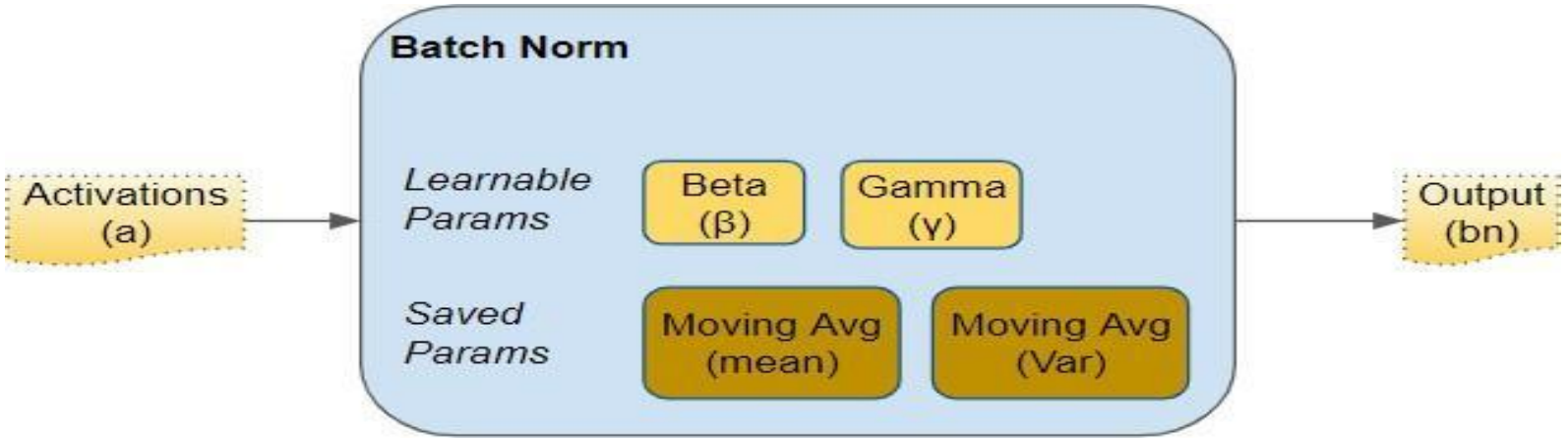
- Use statistics from current batch
- Keep running average of mean and variance (to use later in inference):

$$\mu_{running} = 0.9 \times \mu_{running} + 0.1 \times \mu_{batch}$$

During Inference (We can't depend on mini-batches):

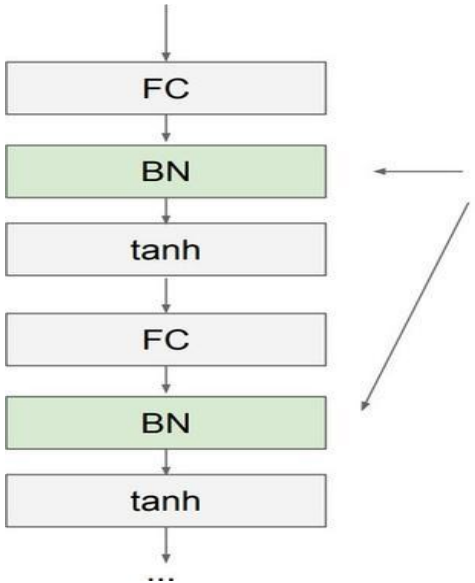
- Use stored running averages
- These are fixed values from training
- No batch statistics needed!

Batch Normalization



[0]Ketan Doshi

Batch Normalization



Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

[0] Ioffe and Szegedy 2015

Batch Normalization

- In Pytorch, batch normalization behavior is controlled by model's mode:

```
# Training Mode:  Use batch statistics model.train()
output = model(input)
# Evaluation Mode:  Use running statistics model.eval()
output = model(input)
```

Batch Normalization

- Advantages:
  - Makes deep networks much easier to train!
  - Improves gradient flow
  - Allows higher learning rates, faster convergence
  - Networks become more robust to initialization
  - Acts as regularization during training
  - Zero overhead at test-time: can be fused with conv!

[0]Slide based on CS231n by Fei-Fei Li, Yunzhu Li & Ruohan Gao

Batch Normalization

- Advantages:
  - Makes deep networks much easier to train!
  - Improves gradient flow
  - Allows higher learning rates, faster convergence
  - Networks become more robust to initialization
  - Acts as regularization during training
  - Zero overhead at test-time: can be fused with conv!
- Disadvantages:
  - Behaves differently during training and testing: this is a very common source of bugs!

[0]Slide based on CS231n by Fei-Fei Li, Yunzhu Li & Ruohan Gao

Full Training Workflow

- **Step 1: Initial Setup**
  - Start with a pretrained model and just finetune when possible; it saves time and improves results.
  - Define an initial architecture without regularization (e.g., dropout) or augmentations.
  - Set up validation strategy and choose an appropriate evaluation loss and metric.
  - Train the model to get a **baseline score**.

Full Training Workflow

- **Step 2: Improvement Process**
  - Overfitting is common at the start; use regularization techniques like:
  - Dropout, batch normalization,...
  - Perform error analysis to identify weaknesses and choose appropriate augmentations or preprocessing.
  - Tune hyperparameters: layers, epochs, learning rate, batch size, etc.
  - Optionally, use ensembling to boost scores (requires more resources).
- **Key Tip:** Track scores and improvements at every step to measure progress effectively.

Full Training Workflow

- **Step 3: Finalization**
  - Save the optimized model for deployment.
  - Use the model for inference in real-world applications.

Image Segmentation

Table of Contents

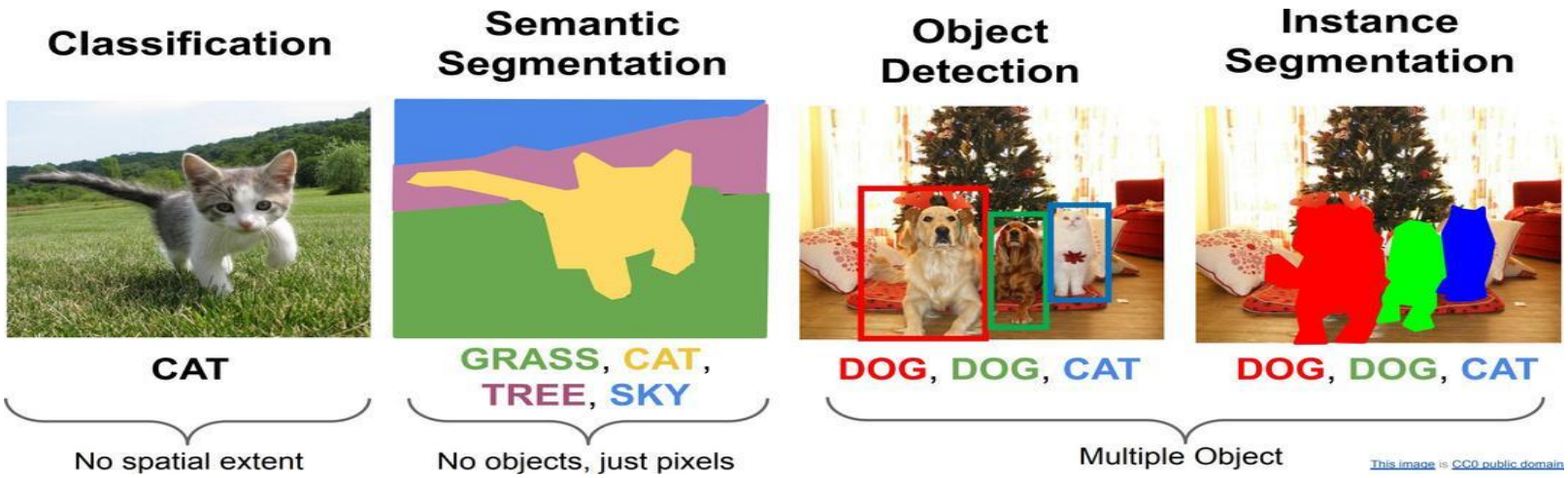
Image Classification

- Previously, we discussed Image Classification
- A core task in Computer Vision

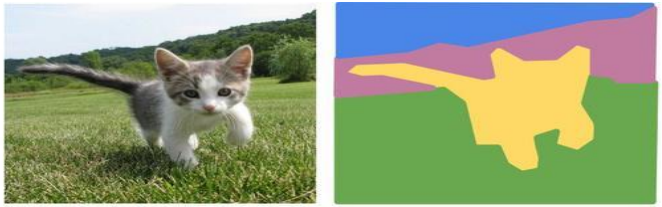
(assume given a set of possible labels)
{dog, cat, truck, plane, ...}

cat

Computer Vision Tasks

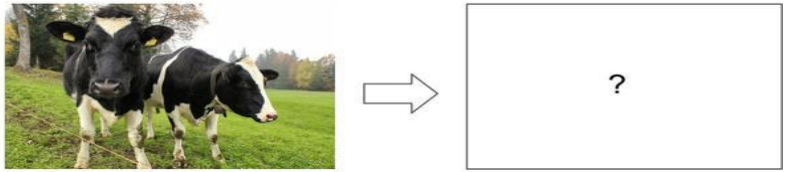


Semantic Segmentation



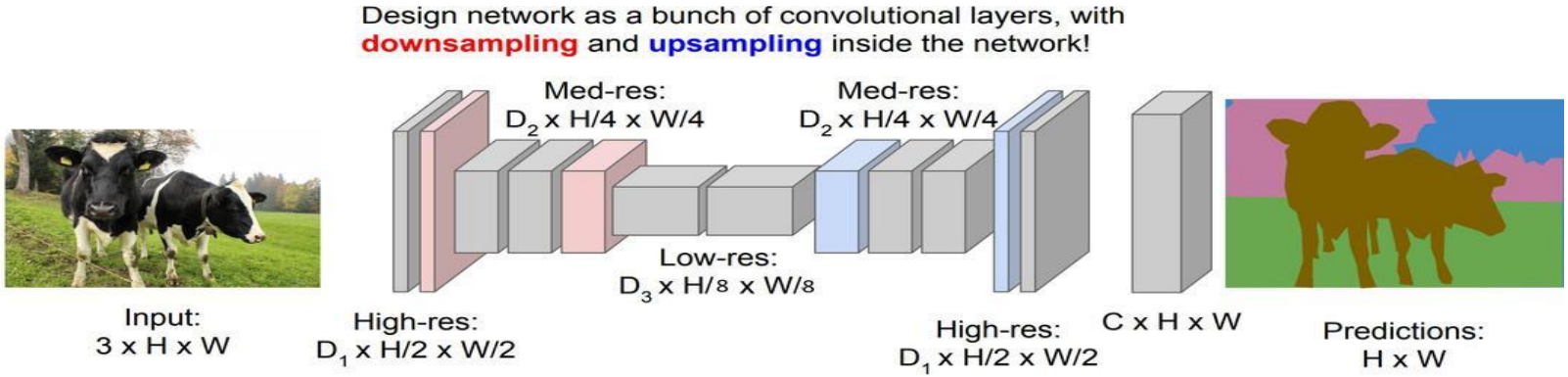
GRASS, CAT,
TREE, SKY, ...

Paired training data: for each training image, each pixel is labeled with a semantic category.



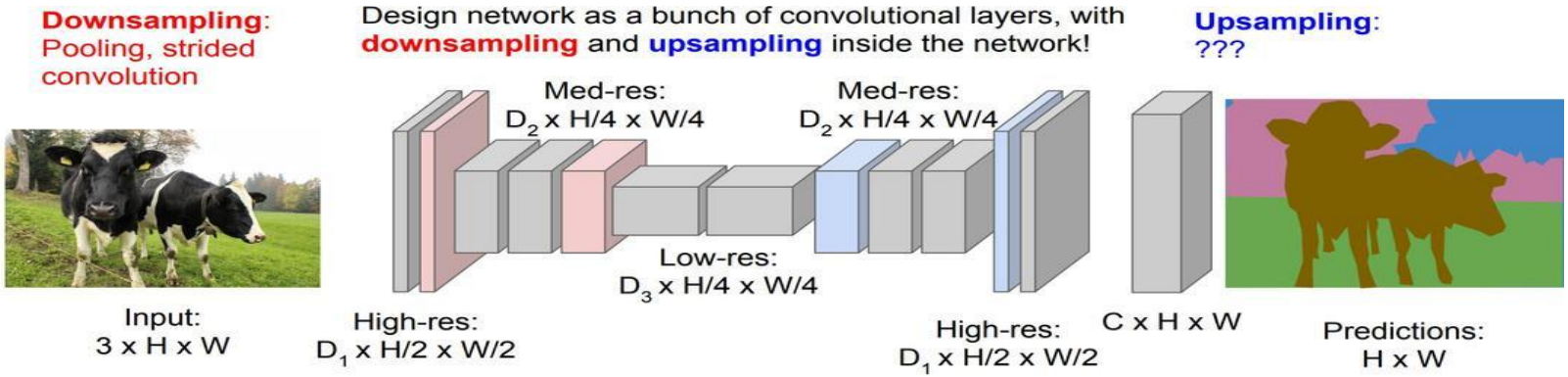
At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Fully Convolutional (cont.)



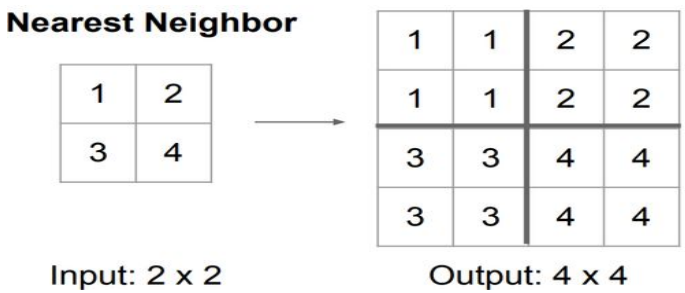
Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

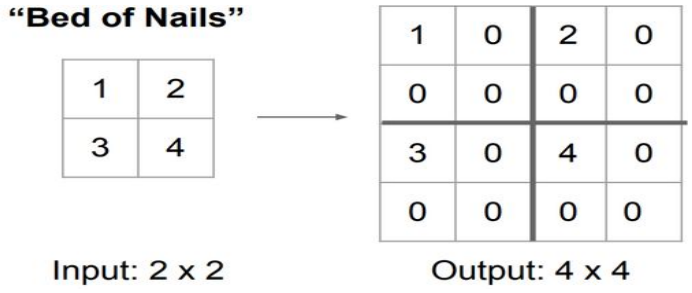
Semantic Segmentation Idea: Fully Convolutional (cont.)



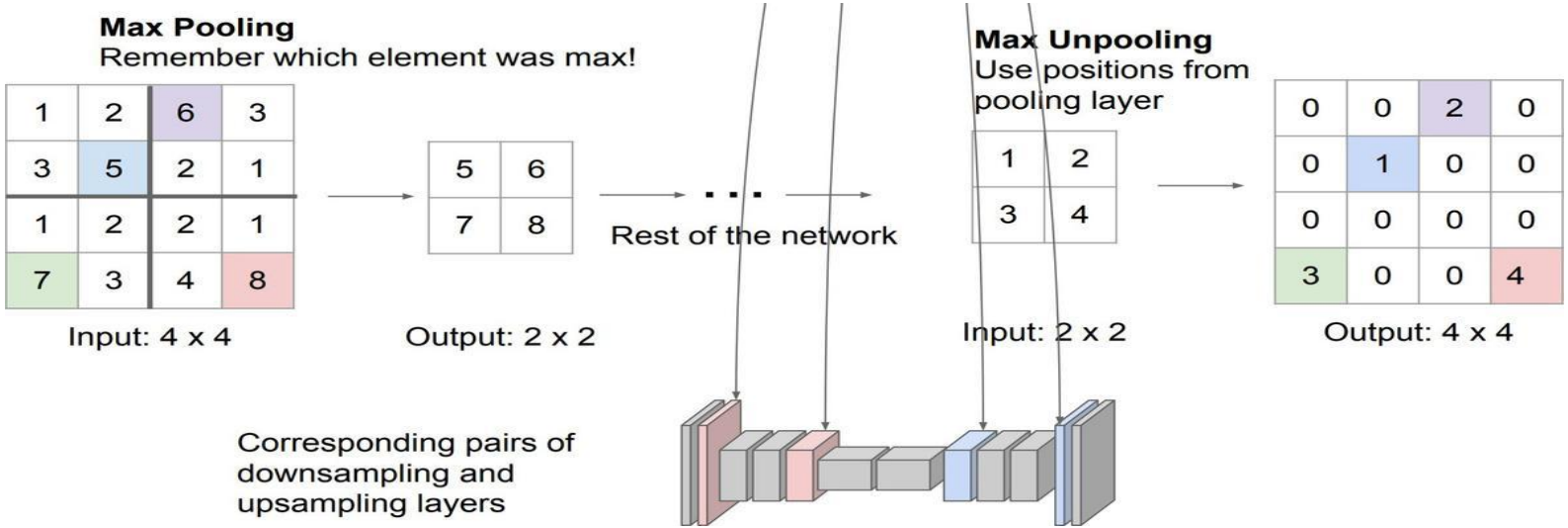
Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

In-Network Upsampling: Unpooling



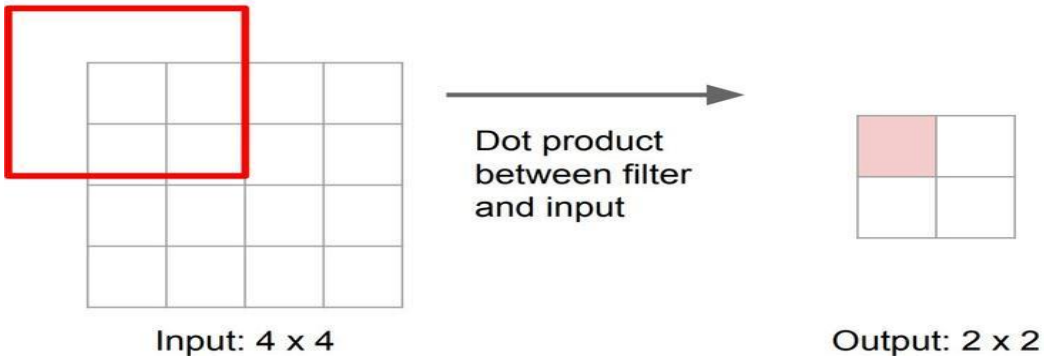


In-Network Upsampling: Max Unpooling



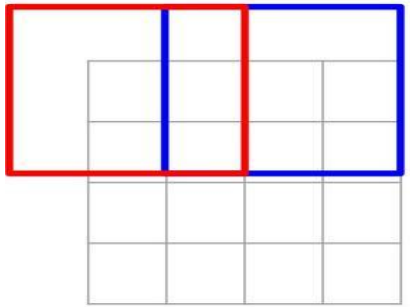
Learnable Upsampling: Transposed Convolution

Recall: Normal 3 x 3 convolution, stride 2 pad 1



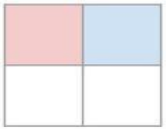
Learnable Upsampling: Transposed Convolution (cont.)

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4

Dot product between filter and input



Output: 2 x 2

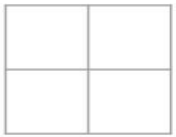
Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

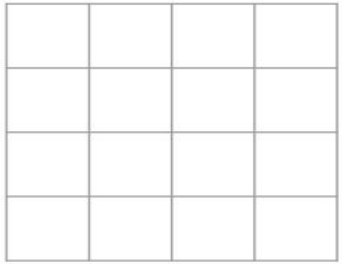
We can interpret strided convolution as “learnable downsampling”.

Learnable Upsampling: Transposed Convolution (cont.)

3 x 3 **transposed** convolution, stride 2 pad 1

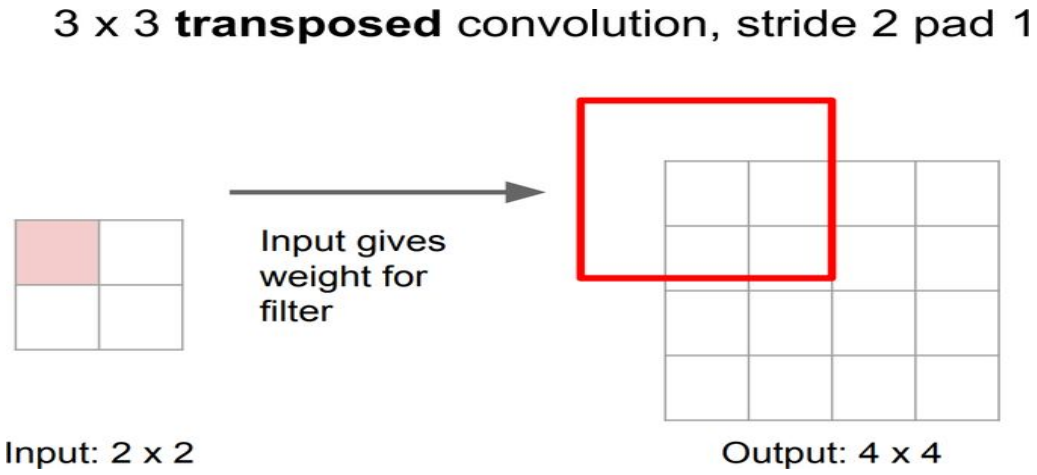


Input: 2 x 2

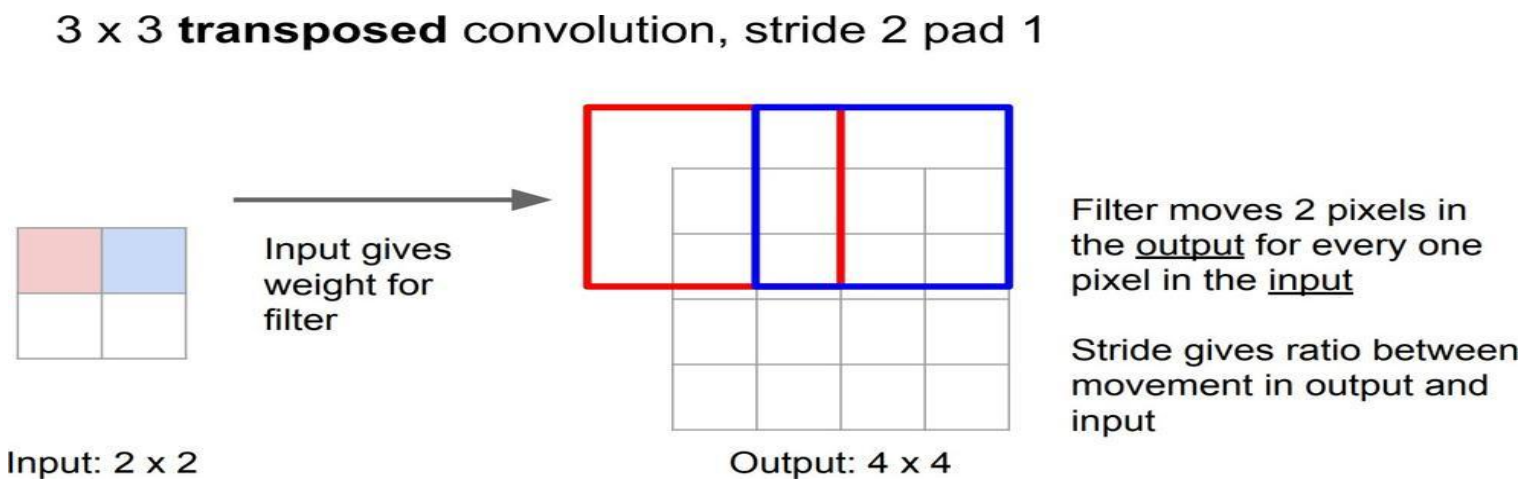


Output: 4 x 4

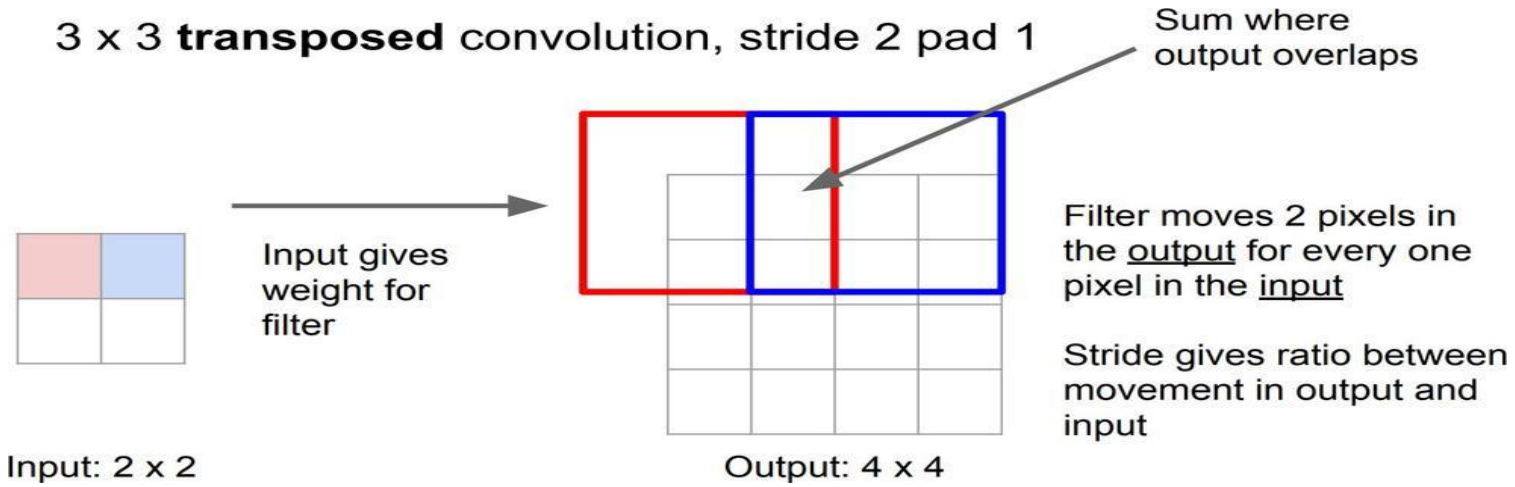
Learnable Upsampling: Transposed Convolution (cont.)



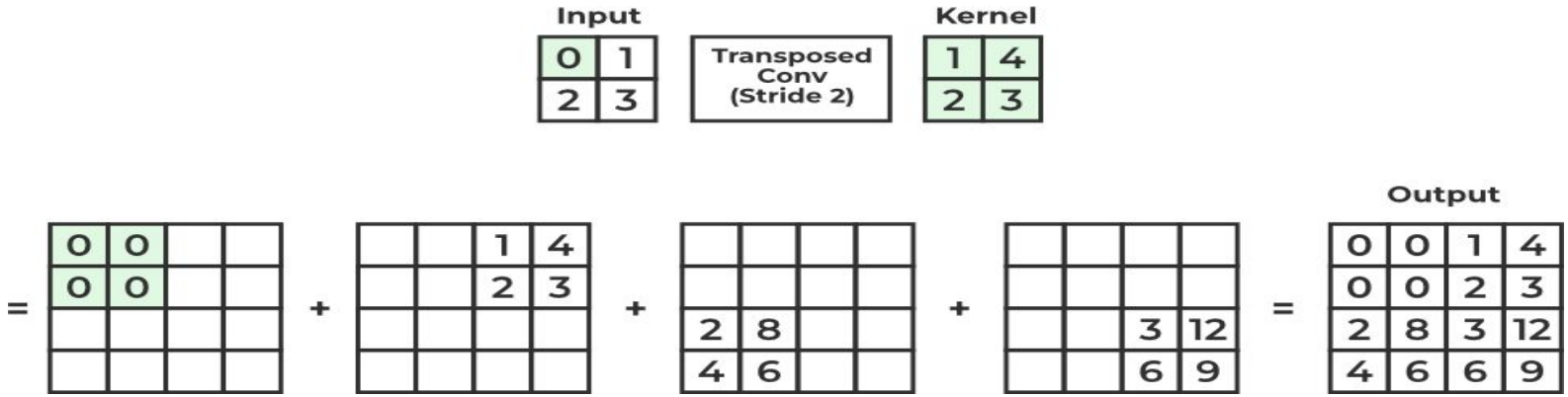
Learnable Upsampling: Transposed Convolution (cont.)



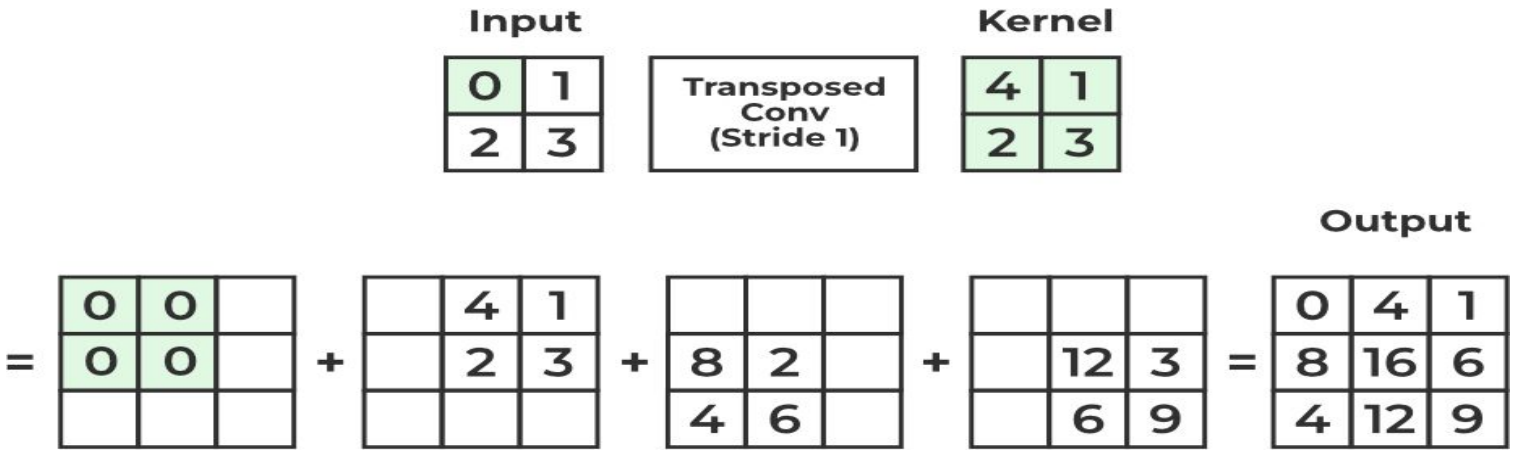
Learnable Upsampling: Transposed Convolution (cont.)



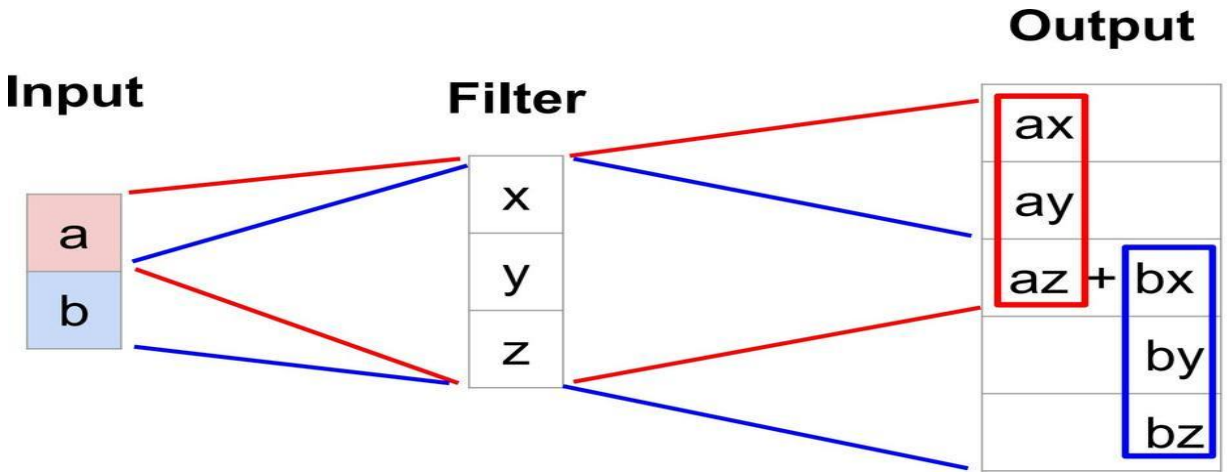
Learnable Upsampling: Transposed Convolution (cont.)



Learnable Upsampling: Transposed Convolution (cont.)



Transposed Convolution: 1D Example



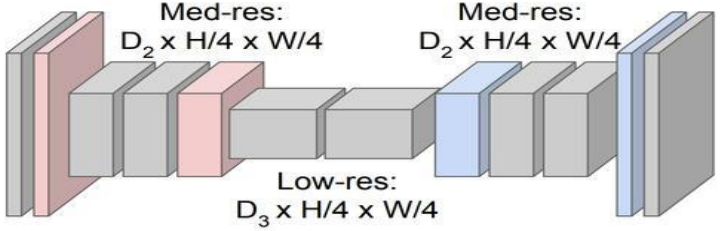
Output contains copies of the filter weighted by the input, summing at where at overlaps in the output

Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided convolution

Input:
3 x H x W

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



High-res:
D_1 x H/2 x W/2

High-res:
D_1 x H/2 x W/2

Upsampling:
Unpooling or strided transposed convolution

Predictions:
H x W

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Can We Do Better?

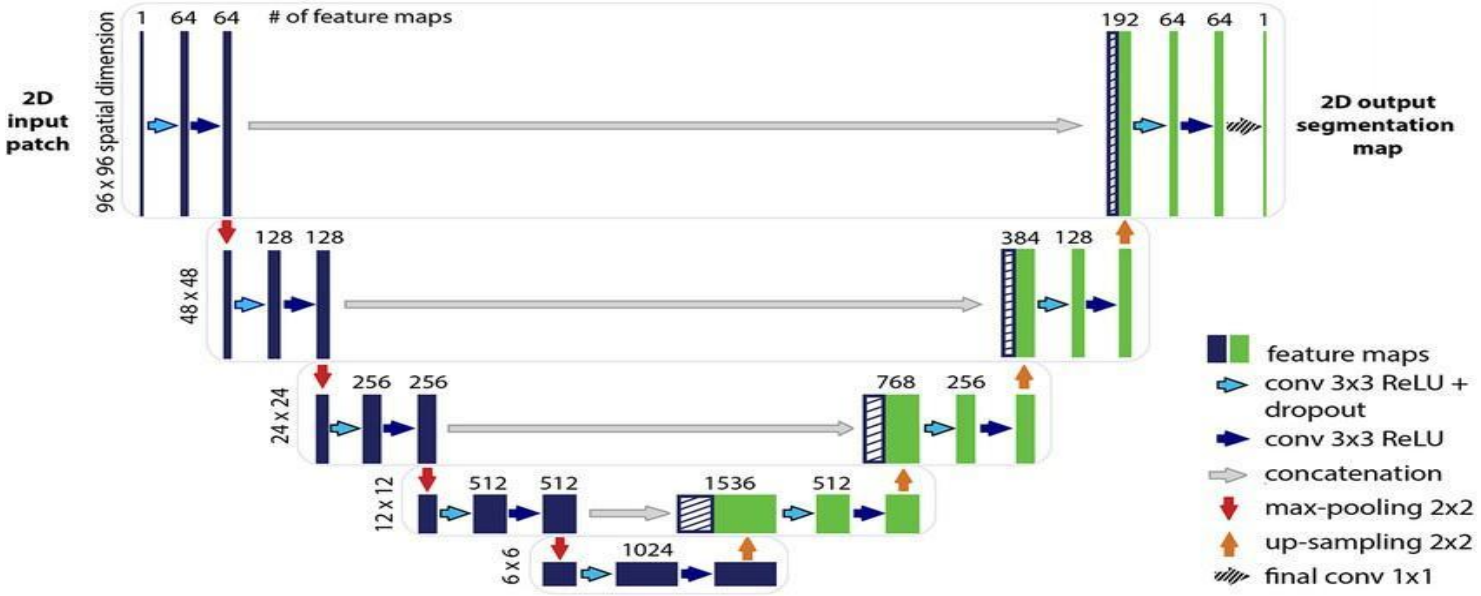
- The downsampling-then-upsampling approach works well for semantic segmentation.
- **But… can we do better?**
- **Problem**: Important details and spatial information may be lost during downsampling.

Can We Do Better?

- The downsampling-then-upsampling approach works well for semantic segmentation.
- **But… can we do better?**
- **Problem**: Important details and spatial information may be lost during downsampling.
- **Solution**: Introduce **residual connections** to preserve spatial information.

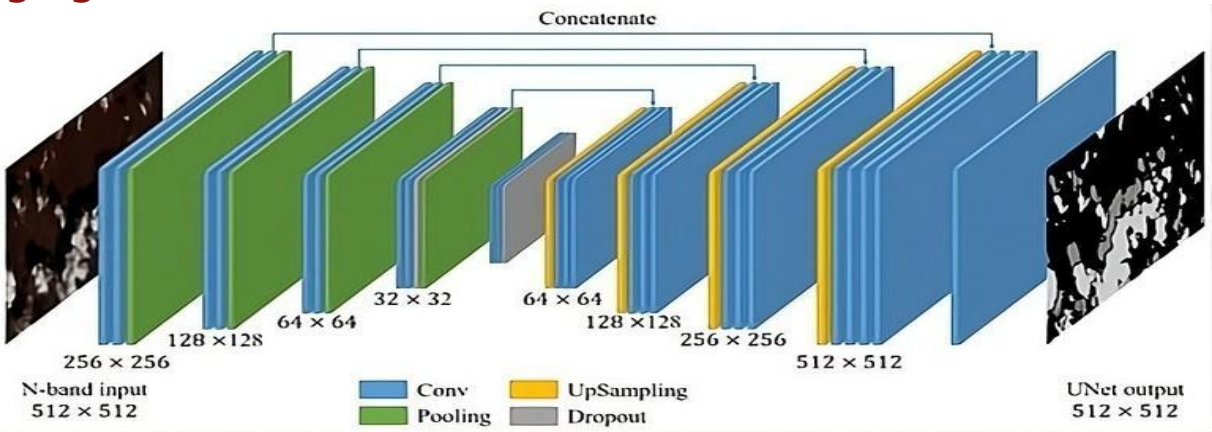
Residual Connections in Segmentation

- Directly connect features from downsampling layers to upsampling layers.
- Help recover lost spatial details and improve segmentation accuracy.



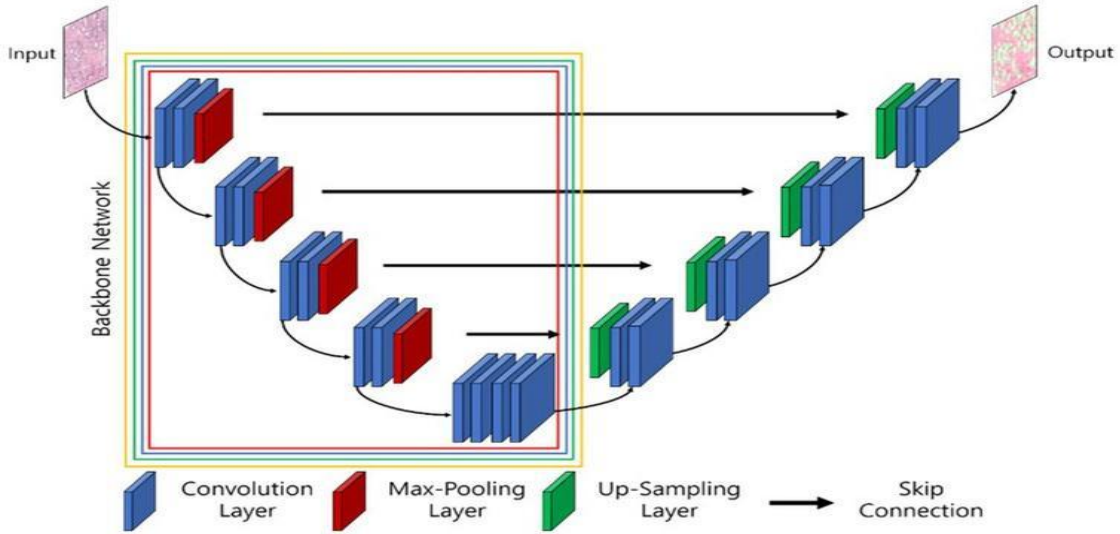
U-Net

- This architecture, with residual connections, is called **U-Net**.
- **Why the name?**
  - The architecture resembles the shape of the letter ”U”.
  - Features are downsampled in the encoder and upsampled in the decoder, with skip connections in between.
- U-Net is widely used for segmentation tasks, especially in biomedical imaging.



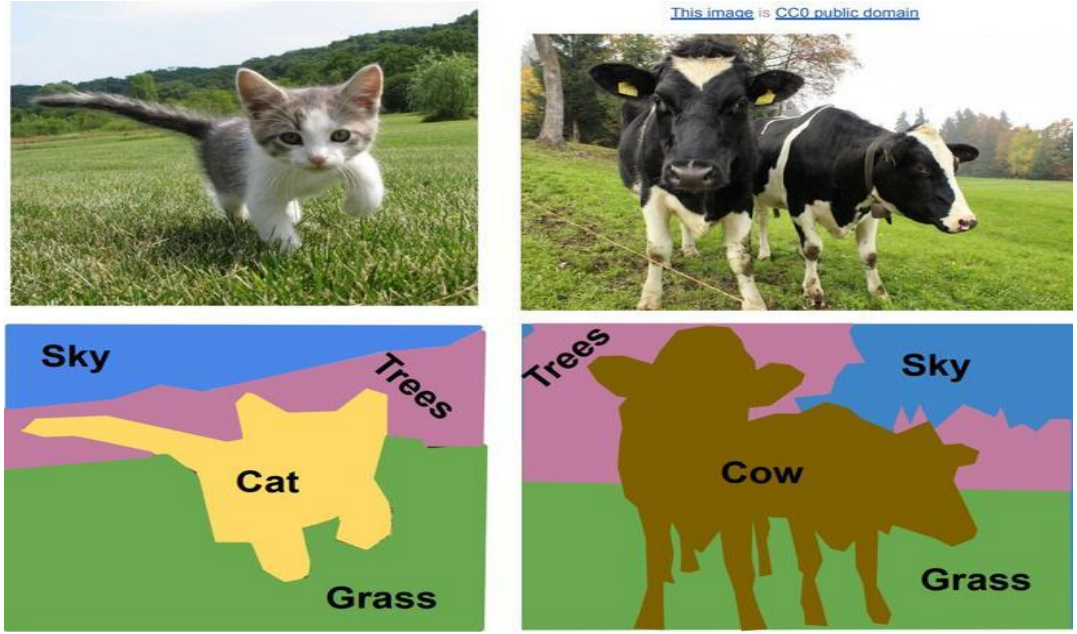
Using a Pretrained U-Net

- **Pretrained Encoder:**
  - The encoder can use a pretrained backbone (e.g., ResNet, EfficientNet).
  - This help utilize features learned on large datasets (e.g., ImageNet).
  - Only the decoder is trained from scratch for segmentation-specific tasks.



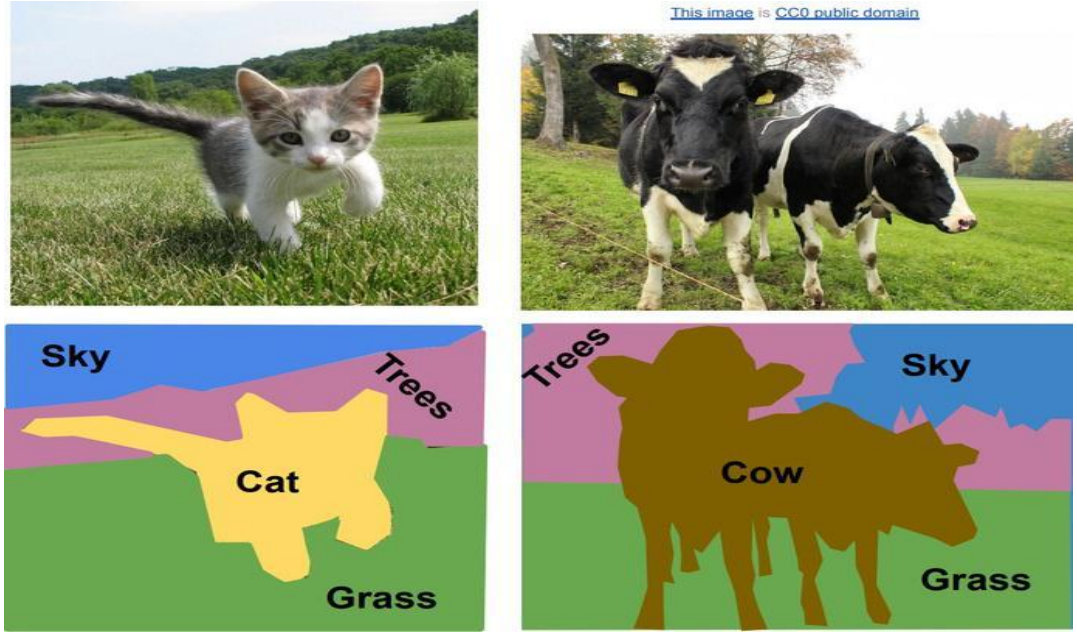
Semantic Segmentation

- Label each pixel in the image with a category label



Semantic Segmentation

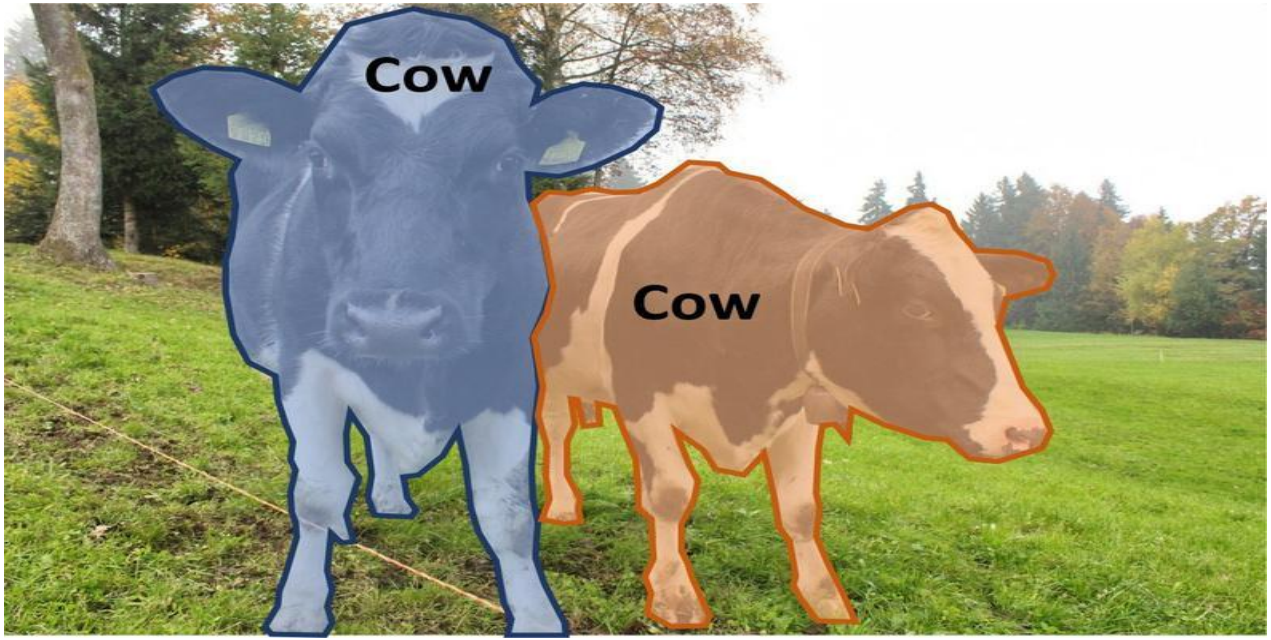
- Label each pixel in the image with a category label



- **Does not differentiate instances, only care about pixels**

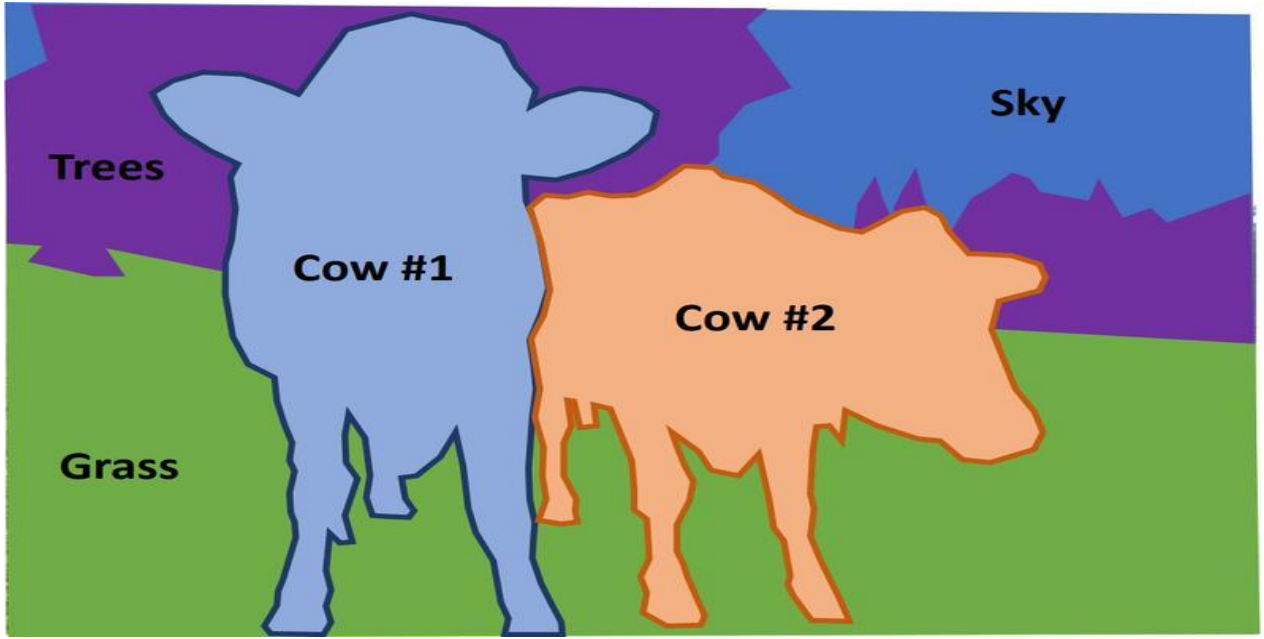
Instance Segmentation

- Separate object instances, but only things



Panoptic Segmentation

- Label all pixels in the image (both things and stuff)



References

These slides have been adapted from

- Fei-Fei Li, Yunzhu Li & Ruohan Gao, Stanford CS231n: Deep Learning for Computer Vision
- Assaf Shocher, Shai Bagon, Meirav Galun & Tali Dekel, WAIC DL4CV Deep Learning for Computer Vision: Fundamentals and Applications
- Justin Johnson, UMich EECS 498.008/598.008: Deep Learning for Computer Vision